
TEAM: Temporal–Spatial Consistency Guided Expert Activation for MoE Diffusion Language Model Acceleration

Linye Wei^{1 2} Zixiang Luo³ Pingzhi Tang^{1 3} Meng Li^{1 2 4}

Abstract

Diffusion large language models (dLLMs) have recently gained significant attention due to their inherent support for parallel decoding. Building on this paradigm, Mixture-of-Experts (MoE) dLLMs with autoregressive (AR) initialization have further demonstrated strong performance competitive with mainstream AR models. However, we identify a fundamental mismatch between MoE architectures and diffusion-based decoding. Specifically, a large number of experts are activated at each denoising step, while only a small subset of tokens is ultimately accepted, resulting in substantial inference overhead and limiting their deployment in latency-sensitive applications. In this work, we propose **TEAM**, a plug-and-play framework that accelerates MoE dLLMs by enabling more accepted tokens with fewer activated experts. TEAM is motivated by the observation that expert routing decisions exhibit strong temporal consistency across denoising levels as well as spatial consistency across token positions. Leveraging these properties, TEAM employs three complementary expert activation and decoding strategies, conservatively selecting necessary experts for decoded and masked tokens and simultaneously performing aggressive speculative exploration across multiple candidates. Experimental results demonstrate that TEAM achieves up to 2.2× speedup over vanilla MoE dLLM, with negligible performance degradation. Code is released at <https://github.com/PKU-SEC-Lab/TEAM-MoE-dLLM>.

1. Introduction

Diffusion large language models (dLLMs) (Nie et al., 2025; Ye et al., 2025; Khanna et al., 2025) address limitations of autoregressive (AR) generation by adopting bidirectional attention, which enables parallelized decoding and positions dLLMs as a compelling alternative to conventional AR models. Recent advances (Wang et al., 2025; Wu et al., 2025; Fu et al., 2025; Liu et al., 2025a) with AR initialization further strengthen this paradigm by incorporating strong autoregressive training priors while remaining compatible with KV cache, achieving both higher accuracy and better inference efficiency than AR models of comparable scale.

Thanks to the advantages in training and inference efficiency as well as their strong scalability, Mixture of Experts (MoE) (Shazeer et al., 2017; Jiang et al., 2024) architectures have become dominant in state-of-the-art AR language models (Yang et al., 2025; Liu et al., 2024; Comanici et al., 2025). Initializing dLLMs from such models, like SDAR (Cheng et al., 2025) and LLaDA 2.0 (Bie et al., 2025), further enhances the effectiveness of diffusion-based decoding, reinforcing the competitiveness of this paradigm compared to the latest AR LLMs (Ma et al., 2025b; Team et al., 2025).

Nevertheless, we observe that naively integrating MoE architectures into dLLMs can substantially degrade inference efficiency. During each diffusion iteration, all tokens within a decoding block are processed in parallel under bidirectional attention, with each token independently selecting its routed experts. However, only a small subset of tokens whose confidence exceeds a predefined threshold are ultimately unmasked. Due to the heterogeneity of expert routing decisions across tokens, a single forward pass typically activates a large fraction of the available experts, leading to significant memory access and communication overhead, while yielding only a limited number of accepted tokens.

As illustrated in Figure 1, for SDAR (Cheng et al., 2025) (8 experts per token), the observed ratio of activated distinct experts to ultimately accepted tokens is substantially higher than eight in practice. This phenomenon highlights the difficulty of applying MoE architectures to dLLMs: it effectively reverts to dense-model inefficiency. While this overhead can be amortized across concurrent requests in cloud deploy-

¹Institute for Artificial Intelligence, Peking University, Beijing, China ²School of Integrated Circuits, Peking University, Beijing, China ³Yuanpei College, Peking University, Beijing, China ⁴Beijing Advanced Innovation Center for Integrated Circuits, Beijing, China. Correspondence to: Meng Li <meng.li@pku.edu.cn>.

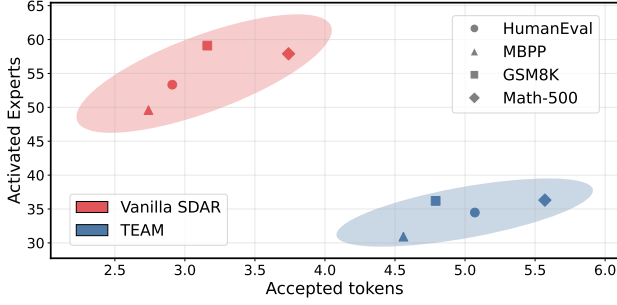


Figure 1. Activated experts vs. accepted tokens per forward pass in SDAR 30B-A3B. TEAM decodes more tokens with fewer experts activated in an iteration.

ments, it becomes a critical bottleneck in scenarios that are highly sensitive to decoding speed and tail latency, as well as on edge platforms with constrained hardware resources.

To address this challenge, we propose **TEAM**, which is developed based on our core observation that although both involve multi token decoding, block-wise inference in dLLMs is fundamentally different from multi batch inference in autoregressive models, exhibiting strong temporal and spatial consistency. **Along the temporal dimension**, dLLMs repeatedly process the same tokens within a block across multiple denoising levels, and for decoded tokens that have already been accepted, their values remain fixed in subsequent iterations within the block, yet they continue to participate in forward computation due to the bidirectional attention mechanism, leading to numerous unnecessary expert activations. **Along the spatial dimension**, the routing of unaccepted, masked tokens is highly concentrated, with relatively little variation in expert selection across tokens. Moreover, owing to the latent causal logic of the decoding process, the acceptance order exhibits spatial locality, suggesting that a substantial portion of masked tokens can be predicted to remain unaccepted in early iterations.

Building on these observations, TEAM implements three customized expert activation strategies for tokens within each block. For **decoded tokens**, we introduce a delayed caching mechanism, activating experts only for recently accepted tokens. For masked tokens, we further partition them into **hot tokens**, which are more likely to be accepted in the near future, and **cold tokens**, which are unlikely to be accepted. Exploiting the concentration of expert routing among masked tokens, TEAM performs speculative exploration on hot tokens to increase the acceptance rate, while rerouting cold tokens to experts that are already activated by decoded or hot tokens. Our main contributions can be summarized as follows:

- We investigate the inefficiency of naively applying

MoE architectures to dLLMs. To the best of our knowledge, this is the first study to specifically analyze expert activation characteristics in MoE dLLMs.

- Leveraging the temporal-spatial consistency of block-wise decoding in dLLMs, we propose TEAM, which implements three complementary expert activation and decoding strategies, achieving more accepted tokens with fewer activated experts.
- Extensive experiments demonstrate that, with our tailored expert activation, TEAM achieves up to 2.2× speedup while preserving model performance.

2. Related work

Diffusion Large Language Models (dLLMs). Autoregressive large language models (LLMs) (Achiam et al., 2023; Grattafiori et al., 2024; Yang et al., 2025) have demonstrated remarkable capabilities across a wide range of applications across a wide range of applications such as text generation, medical science (Zhu et al., 2025a), and embodied AI (Yang et al., 2026b), yet their inference efficiency is constrained by autoregressive decoding. Inspired by the success of diffusion in generative modeling (Ho et al., 2020; Yan et al., 2025; 2026; Zhu et al., 2025b), diffusion large language models (dLLMs) (Nie et al., 2025; Ye et al., 2025; Li et al., 2025a; Yang et al., 2026a; Feng et al., 2026) mitigate this limitation by enabling parallel decoding via bidirectional attention mechanism. In this paradigm, the entire response is represented as masked tokens, and all positions are decoded in each forward pass. Tokens whose confidence exceeds a predefined threshold are accepted, while the remaining tokens are re-masked and refined in subsequent iterations. However, the global bidirectional attention prevented reuse of KV cache, resulting in limited efficiency gains. More recent work (Wang et al., 2025; Tian et al., 2025; Arriola et al., 2025; Gong et al., 2025) adopts a block diffusion strategy, partitioning the response into multiple blocks. By retraining from autoregressive initialization, these models learn to apply bidirectional attention within a block to enable parallel decoding, while using causal attention across blocks to support KV cache. In this way, dLLMs inherit strong autoregressive priors for accuracy while simultaneously improving inference efficiency through parallel decoding and cache reuse.

Mixture-of-Experts (MoE). Unlike dense models which activate all parameters during each forward pass, Mixture-of-Experts (MoE) architectures (Shazeer et al., 2017; Jiang et al., 2024; Xu et al., 2026) replace a single feedforward layer with a group of parallel expert networks and employ a gating mechanism to dynamically select a subset of experts for each token. By enabling expert specialization and sparse parameter activation, MoE architectures scale more effec-

tively to achieve higher model capacity while improving both training and inference efficiency (Huang et al., 2025; Zhong et al., 2025), and have consequently become a dominant design in recent LLMs (Yang et al., 2025; Liu et al., 2024; Comanici et al., 2025). This paradigm has also been extended to dLLMs (Zhu et al., 2025c; Cheng et al., 2025; Bie et al., 2025). However, in dLLMs, all tokens within a block are processed at every diffusion iteration, and each token independently activates its routed experts. As a result, even when the batch size is one, a large fraction of the model’s parameters may be activated in a single forward pass. Such scenarios are often precisely those that are most sensitive to decoding latency, thereby limiting the practical deployment of MoE-based dLLMs.

Acceleration of dLLMs. Several studies have explored techniques to accelerate dLLMs. Early approaches (Wu et al., 2025; Liu et al., 2025b; Ma et al., 2025a) reduce computation through approximate KV caching, but rely on fixed and coarse-grained intervals for cache refresh. Other methods (Chen et al., 2025; Song et al., 2025; Jiang et al., 2025; Qian et al., 2026) limit arithmetic intensity via sparsification, which becomes less critical for block diffusion. A separate line of work (Gao et al., 2025; Agrawal et al., 2025; Wei et al., 2025; Wu & Zhang, 2025; Zhu et al., 2025d) integrates latent refinement or speculative decoding (Chen et al., 2023; Leviathan et al., 2023) into dLLMs to further increase decoding parallelism. Beyond dense models, the strong empirical performance of MoE dLLMs has recently motivated efforts to accelerate this paradigm, such as dInfer (Ma et al., 2025b). However, dInfer primarily targets general dLLM acceleration and focuses only on expert-parallel execution for cloud-scale MoE deployment. In contrast, we present the first dedicated analysis of expert activation behavior in MoE dLLMs and propose TEAM, which exploits temporal-spatial consistency to tailor distinct expert activation strategies for different tokens, thereby improving decoding efficiency.

3. Preliminary and Motivation

The inference process of a dLLM begins by initializing the response Y with $N = B \times L$ [MASK] tokens, where the response is partitioned into B blocks, each containing L tokens. The i -th block is denoted as $Y_i = [y_i^0, y_i^1, \dots, y_i^{L-1}]$. Given a prompt P , the model p_θ performs block-wise decoding, factorizing the response $\hat{Y}$ as:

$$p_\theta(\hat{Y} | P) = \prod_{i=1}^B p_\theta(\hat{Y}_i | P, Y_{\leq i}) \quad (1)$$

Concretely, within each block, the model iteratively samples from the [MASK] tokens. For the i -th block, a single forward pass produces token predictions and corresponding

confidence scores, given by:

$$\begin{aligned} \hat{y}_i^k &= \operatorname{argmax}_{v \in V} p_\theta(y_i^k = v | P, Y_{\leq i}) \text{ and} \\ c_k &= p_\theta(y_i^k = \hat{y}_i^k | P, Y_{\leq i}), k \in [0, 1, \dots, L] \end{aligned} \quad (2)$$

where V denotes the model’s vocabulary. Only tokens whose confidence exceeds a predefined threshold τ are accepted, while the remaining tokens are re-masked for subsequent iterations:

$$y_i^k = \begin{cases} \hat{y}_i^k, & \text{if } c_k > \tau \\ [\text{MASK}], & \text{otherwise} \end{cases}, k \in [0, 1, \dots, L] \quad (3)$$

Once all positions within a block are unmasked, the block is cached and decoding proceeds to the next, repeating this process until the end-of-sequence [EOS] token is generated.

The decoding efficiency of dLLMs arises from the parallel processing of all tokens within a block, which allows multiple tokens to be accepted in a single iteration. However, when this block-wise decoding paradigm is combined with MoE architectures, the parallel tokens collectively activate a large fraction of the experts, negating the benefits of sparse parameter activation. As a result, the combined system can underperform relative to the theoretical gains of its constituent components. To better understand this inefficiency, we analyze the decoding trajectory of a single block along with its associated expert activation patterns, as illustrated in Figure 2, from which we derive several key observations.

Temporal Consistency. Block-wise decoding in diffusion large language models (dLLMs) requires repeatedly processing the same block across successive denoising iterations, during which tokens are gradually accepted and propagated to subsequent steps. Although these accepted tokens no longer change and merely provide context for decoding the remaining masked tokens, they still incur full computation at every iteration and independently trigger expert activations in MoE layers. Figure 2(a) reports the number of experts activated across iterations at three representative layers (first, middle, and last). This repeated computation on already decoded tokens leads to substantial additional expert activations at each step. As decoding progresses and more tokens become fixed, the fraction of total activated experts attributable to these tokens increases, ultimately dominating expert activation in later iterations.

Spatial Consistency. In contrast to the token-specific activations of decoded tokens, variability among masked tokens primarily stems from positional encodings in their input embeddings. Consequently, spatially adjacent masked tokens

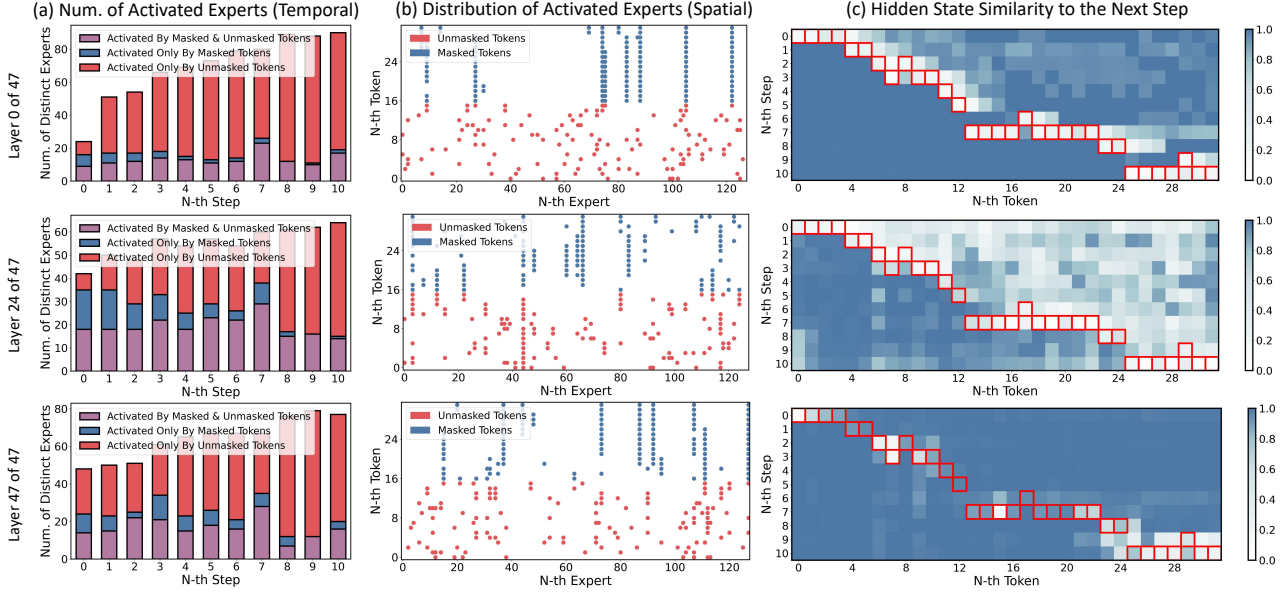


Figure 2. Temporal-spatial characteristics of expert activation and decoding with the SDAR 30B-A3B model on a prompt from the GSM8K dataset. Results are shown for layers 0, 24, and 47 (of 47). (a) Number of activated experts across decoding iterations. (b) Distribution of experts activated by decoded and masked tokens at step 6 (of 11). (c) Token acceptance positions at each iteration, together with hidden state similarity relative to the subsequent iteration. The generality across more datasets is shown in Appendix A.1.

exhibit highly consistent expert routing patterns across layers. As shown in Figure 2(b), while decoded tokens activate a diverse set of experts with an approximately uniform distribution across candidates, masked tokens tend to concentrate their routing decisions on a small subset of experts. This observation suggests that a specific group of experts dominates the decoding of nearly all masked tokens, whereas experts outside this subset contribute only marginally or are invoked by very few tokens. The spatial concentration of expert activation also explains why, in Figure 2(a), the experts activated by masked tokens constitute only a limited fraction of the total activated experts.

Temporal-Spatial Locality. We further analyze the step-wise similarity of hidden states produced by each layer and mark the token positions that are unmasked at each iteration (highlighted by red boxes), as illustrated in Figure 2(c). We find that the most significant change in the hidden state of a token occurs precisely between the iteration in which it is accepted and the subsequent iteration, consistent with observations in prior work (Ma et al., 2025a; Song et al., 2025; Li et al., 2025b). Once a token has been accepted and processed through one additional forward pass, its representation can be regarded as approximately stable. Moreover, the acceptance order within a block largely follows an autoregressive trend over time, and the tokens accepted at each iteration exhibit spatial clustering. This behavior is expected, given that dLLMs are initialized from autoregressive models and that natural language generation inherently follows a causal

structure. As a result, tokens newly accepted at a given step tend to lie close to previously decoded tokens, while some tokens are unlikely to be accepted during early iterations.

Building on these insights, we propose TEAM, which implements three complementary expert activation and decoding strategies tailored to the tokens within each block, as shown in Figure 3.

4. TEAM Methodology

4.1. Delayed Caching for Decoded Tokens (DCD)

As discussed before, once decoded tokens are incorporated as input and processed by an additional forward pass after being accepted, their hidden representations become approximately stable. This observation naturally motivates caching these tokens to avoid redundant computation across iterations. Specifically, at each iteration, computation is performed only for the masked tokens and the tokens newly accepted in the previous step, while the key-value pairs of tokens decoded in earlier iterations are reused from cache. After each forward pass, the KV pairs corresponding to newly accepted tokens are inserted into the cache and reused in subsequent iterations.

A related strategy has been explored in prior work dKV-Cache (Ma et al., 2025a) targeting global bidirectional attention. To mitigate KV drift of decoded tokens under bidirec-

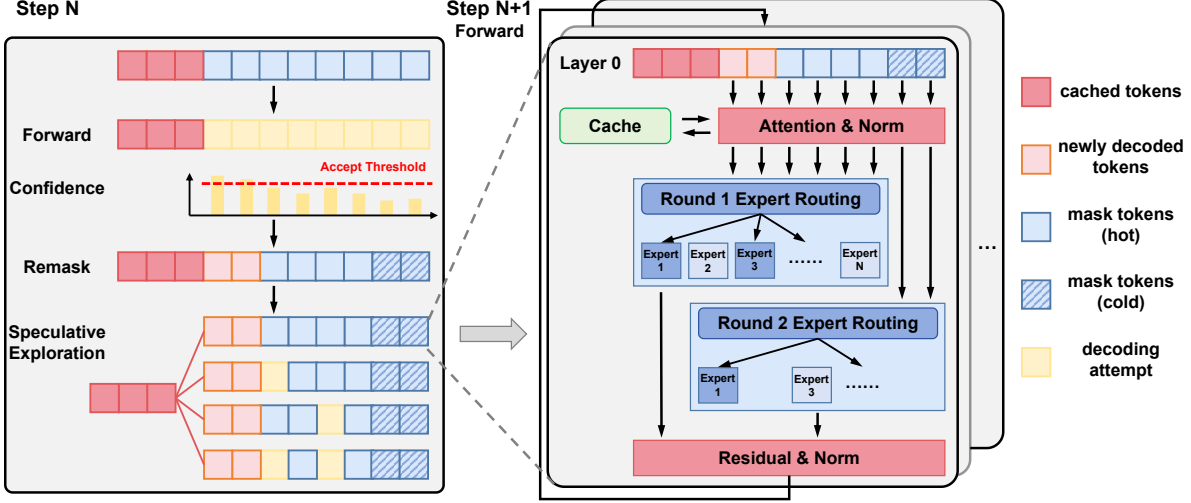


Figure 3. Overview of our proposed **TEAM**. We apply differentiated expert activation and decoding strategies to tokens within each block. For **decoded tokens**, redundant computation is reduced through one-step delayed caching. For **mask tokens (hot)**, we adopt aggressive multi-branch speculative exploration to exploit idle compute resources and increase the token acceptance rate. For **mask tokens (cold)**, a double-round routing mechanism is introduced to constrain unnecessary expert activations.

tional attention, dKV-Cache additionally performs periodic global cache refresh by recomputing all tokens every N iterations. This mechanism becomes less effective in dLLMs that adopt the block diffusion paradigm with native support for KV cache. In this setting, decoding no longer requires global token processing, and parallel computation is confined to tokens within a single block. Such block-level parallelism typically does not reach the compute-bandwidth balance point of modern hardware platforms such as GPUs, leaving decoding largely memory-bound and rendering fine-grained KV caching within a block unnecessary.

However, as observed earlier, under MoE architectures, decoded tokens activate a large set of experts that are largely distinct from those activated by masked tokens, substantially increasing parameter activation density and memory access. This property makes caching decoded tokens particularly beneficial in MoE-based dLLMs. Moreover, by leveraging autoregressive priors and the near-autoregressive acceptance order during decoding, our delayed caching mechanism eliminates the need for periodic global cache refresh. As demonstrated in Section 5, this design improves decoding efficiency without sacrificing model quality.

4.2. Speculative Exploration for Hot Tokens (SEH)

Beyond eliminating redundant expert activations arising from repeatedly processing decoded tokens, we observe that expert activation and decoding for masked tokens are also inefficient. First, each activated expert is typically re-

sponsible for only a small number of tokens, resulting in frequent memory accesses with low computational intensity. Consequently, available compute resources are underutilized, increasing overall decoding latency. Second, the near-autoregressive decoding order and the spatial clustering of accepted tokens indicate that a subset of tokens has a low probability of being accepted in early iterations. Activating experts for such tokens only to remask them afterward leads to wasted computation.

These observations motivate a differentiated computation strategy for masked tokens: we aim to improve decoding efficiency for tokens that are likely to be accepted in the near future, which we refer to as **hot tokens**, while reducing the computational overhead for tokens that are unlikely to be accepted, referred to as **cold tokens**. We identify two characteristics that make masked tokens more likely to be accepted in the next iteration: (1) their decoding attempt y_i^k at the current iteration yields a relatively high confidence score c_k , even if it does not yet exceed the acceptance threshold; and (2) they are spatially closer to decoded tokens, and thus benefit from more informative contextual guidance. Formally, the hot tokens are defined as:

$$y_i^{k-hot} = \{y_i^k \mid (c_k > \tau_h) \text{ or } (\forall j, |k - j| < L_h)\} \quad (4)$$

where τ_h denotes the confidence threshold to identify hot tokens, j indexes the positions of currently decoded tokens, and L_h specifies the maximum allowable distance from

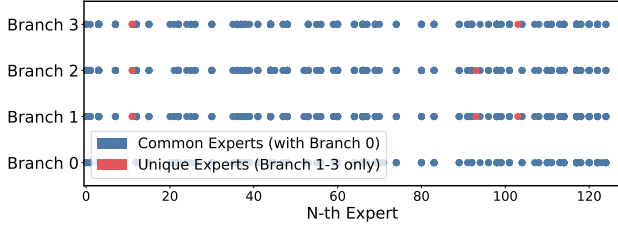


Figure 4. Expert activation with speculative exploration in SDAR for a response from the GSM8K dataset, measured at layer 24 (of 47). More cases are provided in Appendix A.2.

decoded tokens for a masked token to be classified as hot.

For these hot tokens, we employ speculative exploration by additionally accepting the tokens with the top- k confidence scores to construct multiple branches. By decoding and verifying these branches in parallel, we increase the acceptance rate at each iteration, thereby reducing the total number of decoding steps and overall latency. Under bidirectional attention, modifying any single token affects all tokens within the block, implying that each additional candidate effectively incurs the full computational cost of an entire block. For dense models, this behavior is prohibitive, as it rapidly increases computational intensity and shifts the decoding bottleneck from memory-bound to compute-bound, often requiring multi-GPU parallelism to achieve meaningful speedups (Xu et al., 2025). In contrast, for MoE architectures, inference latency is dominated by feedforward layers, and computation is naturally distributed across experts. As illustrated in Figure 4, introducing additional decoding branches increases the originally low arithmetic intensity of each expert, while the high similarity across branches avoids activating a large number of new experts. Consequently, our exploration remains effective for MoE architectures even on a single GPU.

4.3. Limited Activation for Cold Tokens (LAC)

Cold tokens are defined as masked tokens that are both distant from decoded token positions and associated with low confidence scores in previous decoding attempts. Such tokens are unlikely to be accepted in subsequent iterations, implying that activating experts uniquely routed to them is often unnecessary. In practice, decoding attempts for cold tokens have a high probability of being re-masked after each forward pass, rendering their dedicated expert activations largely wasteful.

Leveraging the spatial consistency, masked tokens tend to route to a largely shared subset of experts, with only minor variation across tokens. Based on this property, we apply a limited activation strategy for cold tokens, as summarized

Algorithm 1 Limited Activation (Appendix B)

Input: Decoded tokens D , Mask tokens M , Experts E_0
Output: Activated Experts E_A , Routing Weights W
// 1. Classification of tokens
 Find newly accepted tokens $D_a \subseteq D$
 Find hot tokens $H \subseteq M$ via Eq. 4
 Define cold tokens $C \leftarrow M \setminus H$
// 2. First-round Routing
 Necessary activation $W_1 \leftarrow Router(D_a, H, E_0)$
 Necessary experts $E_A \leftarrow \text{top-}k(W_1)$
// 3. Second-round Routing
 Activation for cold tokens $W_2 \leftarrow Router(C, E_A)$
 Routing Weights $W \leftarrow Concat(W_1, W_2)$
Return $E_A, W=0$

in Algorithm 1. Specifically, we first perform expert routing for newly accepted tokens that have not yet been cached, as well as for hot tokens, since these tokens require accurate expert activation to ensure decoding correctness. The union of experts selected in this stage defines a necessary expert set. We then conduct a second round of routing for cold tokens, restricting their activation to this expert set. Through this double-round routing mechanism, expert activation for cold tokens is strictly confined to necessary experts, avoiding the introduction of token-specific expert activations while preserving model quality and retaining the possibility that cold tokens may still be unexpectedly accepted in later iterations.

5. Experiments

5.1. Experimental Setup

Our experiments are primarily conducted on SDAR 30B-A3B (Cheng et al., 2025), a representative MoE-based diffusion language model that follows the block diffusion paradigm. We adopt the official open-sourced evaluation protocol provided by SDAR as our baseline. We note that for another MoE-based dLLM, LLaDA 2.0 (Bie et al., 2025), an official Hugging Face-format evaluation pipeline is not available, and therefore it is not used as the primary experimental platform in this study. To demonstrate the generality of our proposed TEAM across diverse tasks, we evaluate performance on two code generation benchmarks, HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), as well as two mathematical reasoning benchmarks, GSM8K (Cobbe et al., 2021) and Math-500 (Lightman et al., 2023). All experiments are conducted on an NVIDIA A100 80GB GPU.

Unless otherwise specified, we follow the official implementations of SDAR and LLaDA 2.0. The acceptance threshold for unmasking tokens is set to $\tau = 0.95$, and the block size, which is the number of tokens decoded in parallel at each iteration, is fixed to 32. For masked token classification

Table 1. **Performance of TEAM on SDAR.** APF denotes the number of Activated experts Per Forward pass, TPF denotes accepted Tokens Per Forward pass, and APT denotes the equivalent number of Activated experts Per decoded Token.

Benchmark	Method	Score $\uparrow$	APF $\downarrow$	TPF $\uparrow$	APT $\downarrow$	Speedup
HumanEval (0-shot)	Vanilla	79.27	53.34	2.91	18.33	1 $\times$
	TEAM	79.88 (+0.61)	34.48 (35% $\downarrow$)	5.07 (1.74 $\times$)	6.80 (63% $\downarrow$)	2.20 $\times$
MBPP (0-shot)	Vanilla	65.76	49.59	2.74	18.10	1 $\times$
	TEAM	65.76 (+0.00)	30.92 (38% $\downarrow$)	4.56 (1.66 $\times$)	6.78 (63% $\downarrow$)	2.08 $\times$
GSM8K (0-shot)	Vanilla	90.60	59.11	3.16	18.71	1 $\times$
	TEAM	90.30 (-0.30)	36.20 (39% $\downarrow$)	4.79 (1.52 $\times$)	7.56 (60% $\downarrow$)	1.83 $\times$
Math-500 (0-shot)	Vanilla	76.00	57.90	3.74	15.48	1 $\times$
	TEAM	75.40 (-0.60)	36.31 (37% $\downarrow$)	5.57 (1.49 $\times$)	6.52 (58% $\downarrow$)	1.64 $\times$
Average	Vanilla	77.91	54.99	3.14	17.66	1 $\times$
	TEAM	77.84 (-0.07)	34.48 (37% $\downarrow$)	5.00 (1.59 $\times$)	6.92 (61% $\downarrow$)	1.94 $\times$

Table 2. **Sensitivity Analysis of Hot-Token Hyperparameters.** Model accuracy and the number of Activated experts Per Forward pass (APF) with DCD and LAC strategy on code generation benchmarks.

Benchmark	$\tau_h = 0.4, L_h = 6$		$\tau_h = 0.5, L_h = 5$		$\tau_h = 0.6, L_h = 4$		$\tau_h = 0.7, L_h = 3$		$\tau_h = 0.8, L_h = 2$	
	Score	APF	Score	APF	Score	APF	Score	APF	Score	APF
HumanEval	78.05	24.49	81.09	24.38	79.27	24.01	79.27	23.08	77.44	22.26
MBPP	66.93	22.21	65.76	22.15	68.09	22.12	66.93	21.66	62.65	20.40
Average	72.49	23.35	73.43	23.27	73.68	23.07	73.10	22.37	70.05	21.33

in TEAM, we identify hot tokens either by a confidence threshold $\tau_h = 0.7$ or by requiring them to be no more than $L_h = 3$ positions away from decoded tokens. In speculative exploration for hot tokens, the number of parallel branches is set to 4.

5.2. Main Results

We report the improvements of TEAM over the vanilla model in terms of expert activation and decoding efficiency, as summarized in Table 1. Without TEAM, each layer of the vanilla model activates more than 50 experts on average per forward pass, approaching half of the total 128 experts. However, each forward pass accepts and unmasks only approximately three tokens, leading to more than twice the nominal routing cost (8 experts per token) for decoding a single token, and in the worst case up to 18 activated experts per decoded token. This reveals a fundamental inefficiency: although parallel decoding in dLLMs and sparse parameter activation in MoE architectures are both individually designed to be inference efficient, their naive combination becomes counterproductive. This inherent incompatibility significantly degrades the overall decoding speed.

In contrast, TEAM achieves decoding of more tokens with substantially fewer expert activations through its carefully designed expert activation and decoding strategies, thereby

simultaneously realizing the benefits of parameter sparsity and decoding parallelism. By introducing delayed caching for decoded tokens and limited expert activation for cold tokens, TEAM reduces the number of activated experts by 35–39% across all benchmarks, even after accounting for the additional experts introduced by multi-branch exploration. Moreover, speculative exploration for hot tokens further enhances decoding parallelism, increasing the number of accepted tokens per iteration by 1.49–1.74 $\times$. Benefiting from both higher sparsity and increased parallelism, TEAM requires on average only 6.92 activated experts to decode a single token, which is even lower than the nominal routing cost of 8 experts per token. As a result, TEAM achieves an average speedup of 1.94 $\times$, with a peak speedup of up to 2.2 $\times$ on the HumanEval benchmark.

5.3. Ablation Study and Hyperparameter Sensitivity

To assess the contribution of TEAM to accelerating inference in MoE dLLM, we progressively integrate its core techniques into the vanilla model. As illustrated in Figure 5, we evaluate both the average number of activated experts required to decode a single token and the corresponding speedup throughout this process.

The ablation study show that the introduction of Speculative Exploration for Hot Tokens (SEH) substantially reduces the

Table 3. Performance with Delayed Caching for Decoded Tokens (DCD). Refresh-4 denotes block cache refresh every 4 steps, Refresh-8 denotes block cache refresh every 8 steps, and Refresh-free denotes never refresh (Ours).

Benchmark	Refresh-4			Refresh-8			Refresh-free		
	Score	APF	Speedup	Score	APF	Speedup	Score	APF	Speedup
HumanEval	79.88	32.67	1.38×	78.66	29.61	1.47×	79.88	26.72	1.58×
MBPP	66.15	29.89	1.32×	66.15	26.91	1.44×	65.76	23.67	1.55×
GSM8K	90.52	35.45	1.27×	90.60	31.23	1.35×	90.45	27.99	1.44×
Math-500	74.80	35.25	1.17×	73.00	31.18	1.25×	74.20	27.52	1.32×
Average	77.84	33.32	1.29×	77.10	29.73	1.38×	77.57	26.48	1.47×

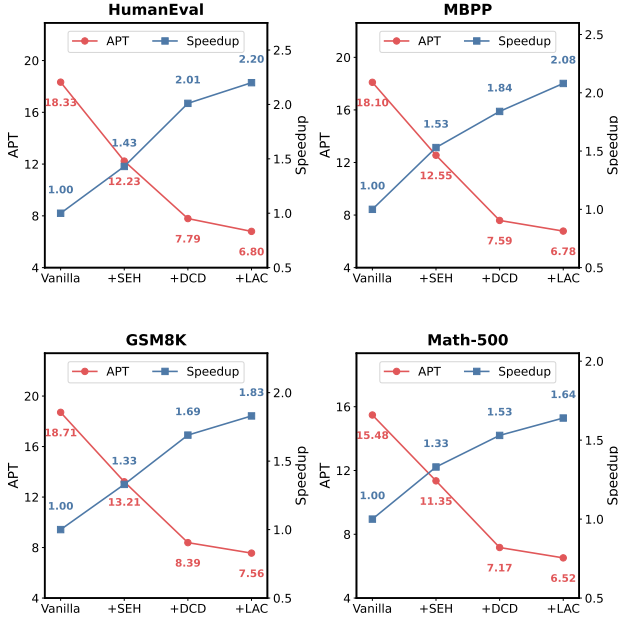


Figure 5. Ablation study on the Activated experts Per decoded Token (APT) and speedup compared to the vanilla model.

number of activated experts per decoded token, validating its effectiveness. By decreasing the number of decoding steps, SEH increases the token acceptance rate per forward pass, while incurring only a marginal increase in additional expert activations thanks to the similarity across multiple branches. Delayed Caching for Decoded Tokens (DCD) further eliminates a large fraction of expert activations triggered by already decoded tokens that are irrelevant to the decoding process itself and only provide contextual guidance. Finally, Limited Activation for Cold Tokens (LAC) strictly confines expert activation to the subset responsible for newly decoded tokens and hot tokens. This design further reduces the number of activated experts per decoded token and yields additional speedup, resulting in the highest overall decoding efficiency among all benchmarks.

Furthermore, we will discuss the hyperparameter selection in our proposed TEAM.

A key factor in accelerating MoE dLLMs with TEAM is the criterion used to classify masked tokens into hot and cold categories, which is governed by a confidence threshold τ_h and a distance constraint L_h . We study the impact of different combinations of these two hyperparameters on model accuracy and the number of activated experts per forward pass on two code generation benchmarks, HumanEval and MBPP, as summarized in Table 2. As τ_h increases up to 0.7 and L_h decreases to 3, the model preserves its accuracy while the number of activated experts decreases monotonically, indicating improved efficiency. Further narrowing the set of tokens identified as hot leads to noticeable performance degradation. Consequently, the configuration $\tau_h = 0.7$ and $L_h = 3$ achieves the best accuracy-efficiency trade-off and is adopted as the default setting in TEAM.

5.4. Analysis of Key Design Choices in TEAM

Refresh-free Caching for Decoded Tokens. As discussed in Section 4, caching decoded tokens at every iteration like dKV-Cache yields limited acceleration for dLLMs that adopt the block diffusion paradigm. However, reducing expert activations through caching is critical to the efficiency of MoE dLLMs. Unlike prior work that periodically refreshes cached representations, we observe that for dLLMs initialized from autoregression, the representations of previously decoded tokens remain highly stable across diffusion iterations, rendering cache refresh unnecessary. As shown in Table 3, we evaluate different refresh intervals, including recomputing all tokens every 4 steps, every 8 steps, or disabling refresh entirely within a block. We find no evidence that eliminating cache refresh leads to a noticeable degradation in model performance. In fact, the Refresh-free configuration even achieves slightly higher accuracy than Refresh-8. In contrast, the efficiency gains from removing refresh operations are substantial, making refresh-free caching a clear advantage for MoE dLLM inference.

Aligned Token Candidates in SEH. To fully exploit the

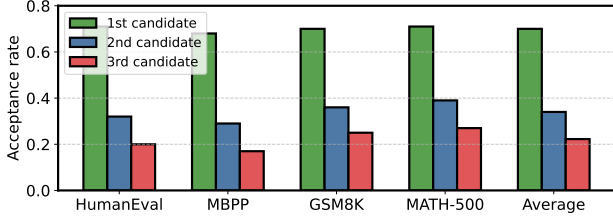


Figure 6. Acceptance probability of the top3-confidence candidate tokens in the subsequent iteration in SEH.

idle compute resources on GPU platforms, we introduce speculative exploration through additional acceptance of candidate tokens. Specifically, three extra branches are constructed by additionally accepting the 1st-confidence token, the 2nd-confidence token, and both tokens simultaneously, even when their confidence scores do not exceed the acceptance threshold. We argue that this design is preferable to further accepting the 3rd-confidence token. To justify this choice, we analyze the acceptance probability of the top3-confidence candidate tokens in the subsequent iteration, as illustrated in Figure 6. The 2nd candidate has a relatively low probability of being directly accepted, while the 3rd one is rarely accepted. This indicates that constructing more diverse branches is inefficient. In contrast, aligned token combinations enable chained verification, thereby improving efficiency.

The Shared Expert Subset for Masked Tokens. A key observation of our proposed TEAM is that a specific subset of experts dominates the decoding process for most masked tokens, while the remaining experts contribute only marginally or are rarely activated. We further investigate the underlying reason for this phenomenon and provide evidence to support it. The dominance arises from the high similarity among model inputs for masked tokens. All masked tokens share the same [M] symbol and are mapped to identical token embeddings. The only source of variation comes from positional encodings, which are relatively minor within a block. As shown in Table 4, we compute the average pairwise cosine similarity of hidden states among all masked tokens within a block at each decoding layer, and then report the highest and the average similarity in all layers. Such a high degree of similarity indicates that this homogeneity persists throughout the entire decoding process, leading to similar routing decisions.

6. Conclusion

In this paper, we propose **TEAM**, a framework for accelerating MoE diffusion language models through temporal and spatial consistency guided expert activation. Through

Table 4. Highest and average pairwise cosine similarity of hidden states among all masked tokens within a block across different layers.

	HumanEval	MBPP	GSM8K	MATH-500
Highest	0.99	0.98	0.99	0.99
Average	0.86	0.84	0.86	0.86

a systematic analysis of temporal consistency across denoising iterations and spatial consistency across token positions, TEAM introduces three complementary expert activation and decoding strategies tailored to different subsets of tokens within each decoding block, enabling more tokens to be decoded with fewer activated experts. Compared to vanilla inference, TEAM achieves up to a 2.2 $\times$ speedup, demonstrating an efficient and practical integration of dLLM and MoE architectures.

Acknowledgments

This work was supported in part by the National Key Research and Development Program under Grant 2024YFB4505004, in part by NSFC under Grant 62495102, Grant 92464104, and Grant 62341407, in part by Beijing Municipal Science and Technology Program under Grant Z241100004224015, in part by 111 Project under Grant B18001.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Agrawal, S., Garrepalli, R., Goel, R., Lee, M., Lott, C., and Porikli, F. Spiffy: Multiplying diffusion llm acceleration via lossless speculative decoding. *arXiv preprint arXiv:2509.18085*, 2025.
- Arriola, M., Gokaslan, A., Chiu, J. T., Yang, Z., Qi, Z., Han, J., Sahoo, S. S., and Kuleshov, V. Block diffusion: Interpolating between autoregressive and diffusion language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al.

- Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Bie, T., Cao, M., Chen, K., Du, L., Gong, M., Gong, Z., Gu, Y., Hu, J., Huang, Z., Lan, Z., Li, C., Li, C., Li, J., Li, Z., Liu, H., Liu, L., Lu, G., Lu, X., Ma, Y., Tan, J., Wei, L., Wen, J.-R., Xing, Y., Zhang, X., Zhao, J., Zheng, D., Zhou, J., Zhou, J., Zhou, Z., Zhu, L., and Zhuang, Y. Llada2.0: Scaling up diffusion language models to 100b, 2025. URL <https://arxiv.org/abs/2512.15745>.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. 2021.
- Chen, X., Huang, S., Guo, C., Wei, C., He, Y., Zhang, J., Li, H., Chen, Y., et al. Dpad: Efficient diffusion language models with suffix dropout. *arXiv preprint arXiv:2508.14148*, 2025.
- Cheng, S., Bian, Y., Liu, D., Zhang, L., Yao, Q., Tian, Z., Wang, W., Guo, Q., Chen, K., Qi, B., et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Feng, S., Chen, Z., Ma, X., Fang, G., and Wang, X. dvoting: Fast voting for dllms. *arXiv preprint arXiv:2602.12153*, 2026.
- Fu, Y., Whalen, L., Ye, Z., Dong, X., Diao, S., Liu, J., Wu, C., Zhang, H., Xie, E., Han, S., et al. Efficient-dlm: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025.
- Gao, Y., Ji, Z., Wang, Y., Qi, B., Xu, H., and Zhang, L. Self speculative decoding for diffusion large language models. *arXiv preprint arXiv:2510.04147*, 2025.
- Gong, S., Agarwal, S., Zhang, Y., Ye, J., Zheng, L., Li, M., An, C., Zhao, P., Bi, W., Han, J., Peng, H., and Kong, L. Scaling diffusion language models via adaptation from autoregressive models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=j1tSLYKwg8>.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Huang, H., Zhong, S., Zhang, Z., Li, S., Niu, D., Zheng, H., Wang, R., and Li, M. Hd-moe: Hybrid and dynamic parallelism for mixture-of-expert llms with 3d near-memory processing. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pp. 1–9. IEEE, 2025.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Jiang, Y., Cai, Y., Luo, X., Fu, J., Wang, J., Liu, C., and Yang, X. d² cache: Accelerating diffusion-based llms via dual adaptive caching. *arXiv preprint arXiv:2509.23094*, 2025.
- Khanna, S., Kharbanda, S., Li, S., Varma, H., Wang, E., Birnbaum, S., Luo, Z., Miraoui, Y., Palrecha, A., Ermon, S., et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.

- Li, J.-N., Guan, J., Wu, W., and Li, C. Refusion: A diffusion large language model with parallel autoregressive decoding. *arXiv preprint arXiv:2512.13586*, 2025a.
- Li, S., Gu, J., Liu, K., Lin, Z., Wei, Z., Grover, A., and Kuen, J. Sparse-lavida: Sparse multimodal discrete diffusion language models. *arXiv preprint arXiv:2512.14008*, 2025b.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Liu, A., He, M., Zeng, S., Zhang, S., Zhang, L., Wu, C., Jia, W., Liu, Y., Zhou, X., and Zhou, J. Wedlm: Reconciling diffusion language models with standard causal attention for fast inference. *arXiv preprint arXiv:2512.22737*, 2025a.
- Liu, Z., Yang, Y., Zhang, Y., Chen, J., Zou, C., Wei, Q., Wang, S., and Zhang, L. dllm-cache: Accelerating diffusion large language models with adaptive caching. *arXiv preprint arXiv:2506.06295*, 2025b.
- Ma, X., Yu, R., Fang, G., and Wang, X. dkv-cache: The cache for diffusion language models. *arXiv preprint arXiv:2505.15781*, 2025a.
- Ma, Y., Du, L., Wei, L., Chen, K., Xu, Q., Wang, K., Feng, G., Lu, G., Liu, L., Qi, X., et al. dinfer: An efficient inference framework for diffusion language models. *arXiv preprint arXiv:2510.08666*, 2025b.
- Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Qian, Y.-Y., Su, J., Hu, L., Zhang, P., Deng, Z., Zhao, P., and Zhang, H. d3llm: Ultra-fast diffusion llm using pseudo-trajectory distillation. *arXiv preprint arXiv:2601.07568*, 2026.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Song, Y., Liu, X., Li, R., Liu, Z., Huang, Z., Guo, Q., He, Z., and Qiu, X. Sparse-dllm: Accelerating diffusion llms with dynamic cache eviction. *arXiv preprint arXiv:2508.02558*, 2025.
- Team, L., Li, A., Liu, B., Hu, B., Li, B., Zeng, B., Ye, B., Tang, C., Tian, C., Huang, C., et al. Every activation boosted: Scaling general reasoner to 1 trillion open language foundation. *arXiv preprint arXiv:2510.22115*, 2025.
- Tian, Y., Liang, Y., Sun, J., Zhang, S., Yang, G., Shu, Y., Fang, S., Guo, T., Han, K., Xu, C., et al. From next-token to next-block: A principled adaptation path for diffusion llms. *arXiv preprint arXiv:2512.06776*, 2025.
- Wang, X., Xu, C., Jin, Y., Jin, J., Zhang, H., and Deng, Z. Diffusion llms can do faster-than-ar inference via discrete diffusion forcing. *arXiv preprint arXiv:2508.09192*, 2025.
- Wei, L., Chen, W., Tang, P., Guo, X., Ye, L., Wang, R., and Li, M. Orchestrating dual-boundaries: An arithmetic intensity inspired acceleration framework for diffusion language models. *arXiv preprint arXiv:2511.21759*, 2025.
- Wu, C., Zhang, H., Xue, S., Diao, S., Fu, Y., Liu, Z., Molchanov, P., Luo, P., Han, S., and Xie, E. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*, 2025.
- Wu, S. and Zhang, J. Free draft-and-verification: Toward lossless parallel decoding for diffusion large language models. *arXiv preprint arXiv:2510.00294*, 2025.
- Xu, C., Jin, Y., Li, J., Tu, Y., Long, G., Tu, D., Hou, T., Yan, J., and Deng, Z. Lopa: Scaling dllm inference via look-ahead parallel decoding. *arXiv preprint arXiv:2512.16229*, 2025.
- Xu, Y., Hu, R., Liu, Z., Sun, M., and Yuan, K. Grouter: Decoupling routing from representation for accelerated moe training. *arXiv preprint arXiv:2603.06626*, 2026.
- Yan, R., Cheng, J., Gan, Y., Sun, S., Wu, Y., Yang, Y., Ling, L., Lin, J., Zhu, Y., Zhou, J., et al. Entropy-adaptive diffusion policy optimization with dynamic step alignment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1924–1934, 2025.
- Yan, R., Cheng, J., Sun, S., Sun, Y., Wu, Y., Peng, W., Wang, Z., Liang, L., Xing, J., and Cai, Y. Do less, achieve more: Do we need every-step optimization for rl fine-tuning of diffusion models? *arXiv preprint arXiv:2605.15855*, 2026.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang,

- T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, K., Teoh, J., Yang, K., Zhang, Y., and Lamb, A. Improving sampling for masked diffusion models via information gain. *arXiv preprint arXiv:2602.18176*, 2026a.
- Yang, Z., Zheng, S., Xie, T., Xu, T., Yu, B., Wang, F., Tang, J., Liu, S., and Li, M. Efficientnav: Towards on-device object-goal navigation with navigation map caching and retrieval. *Advances in Neural Information Processing Systems*, 38:4286–4312, 2026b.
- Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., and Kong, L. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Zhong, S., Sun, Y., Liang, L., Wang, R., Huang, R., and Li, M. Hybrimoe: Hybrid cpu-gpu scheduling and cache management for efficient moe inference. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pp. 1–7. IEEE, 2025.
- Zhu, C., Lin, Y., Shao, J., Lin, J., and Wang, Y. Pathology-aware prototype evolution via llm-driven semantic disambiguation for multicenter diabetic retinopathy diagnosis. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 9196–9205, 2025a.
- Zhu, C., Shao, J., Lin, J., Wang, Y., Wang, J., Tang, J., and Li, K. fmri2ges: Co-speech gesture reconstruction from fmri signal with dual brain decoding alignment. *IEEE Transactions on Circuits and Systems for Video Technology*, 2025b.
- Zhu, F., You, Z., Xing, Y., Huang, Z., Liu, L., Zhuang, Y., Lu, G., Wang, K., Wang, X., Wei, L., et al. Llada-moe: A sparse moe diffusion language model. *arXiv preprint arXiv:2509.24389*, 2025c.
- Zhu, Q., Yao, Y., Zhao, R., Xiang, Y., Saseendran, A., Jin, C., Teare, P., Liang, B., He, Y., and Gui, L. Latent refinement decoding: Enhancing diffusion-based language models by refining belief states. *arXiv preprint arXiv:2510.11052*, 2025d.

A. More details of Temporal-spatial characteristics in dLLMs

A.1. Temporal-spatial characteristics of expert activation and decoding

To demonstrate the generality of our observations, this section presents the temporal-spatial characteristics of expert activation and decoding in the SDAR 30B-A3B model across prompts from diverse benchmarks. We find that MoE-based diffusion language models consistently exhibit strong temporal consistency across denoising iterations and spatial consistency across token positions, which directly motivates the design of our proposed TEAM.

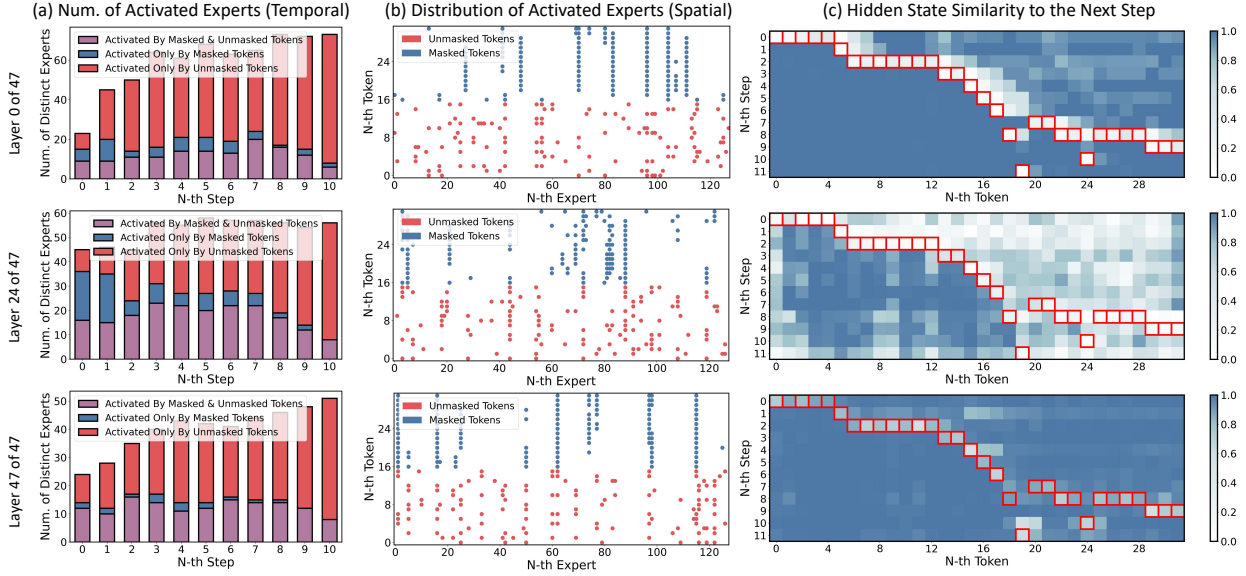


Figure 7. Temporal-spatial characteristics of expert activation and decoding with the SDAR 30B-A3B model on a prompt from the HumanEval dataset.

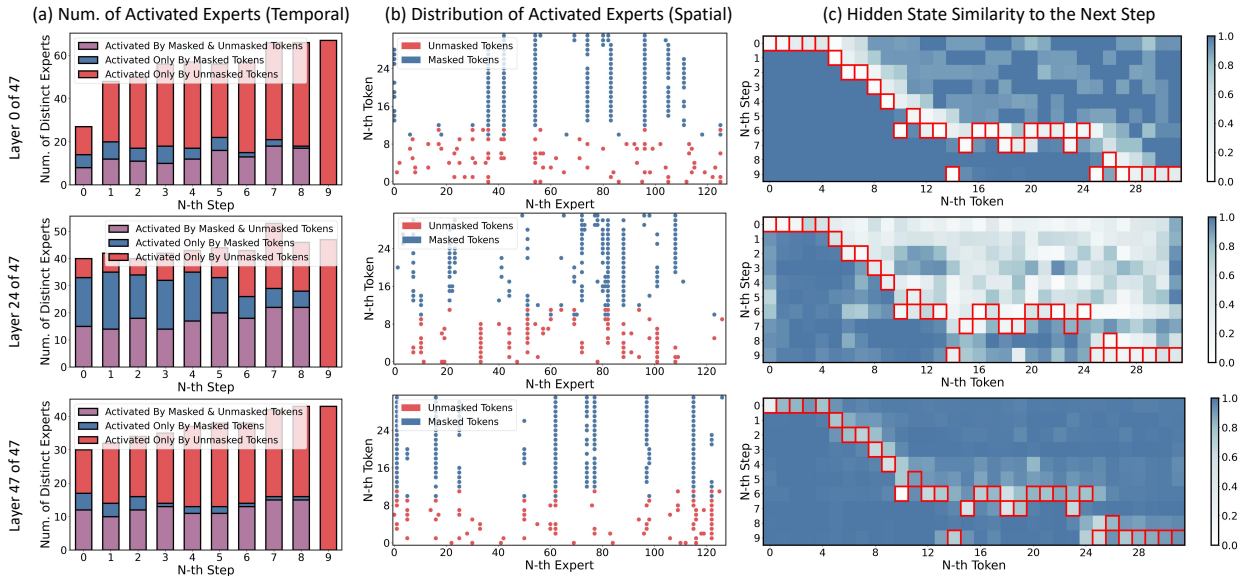


Figure 8. Temporal-spatial characteristics of expert activation and decoding with the SDAR 30B-A3B model on a prompt from the MBPP dataset.

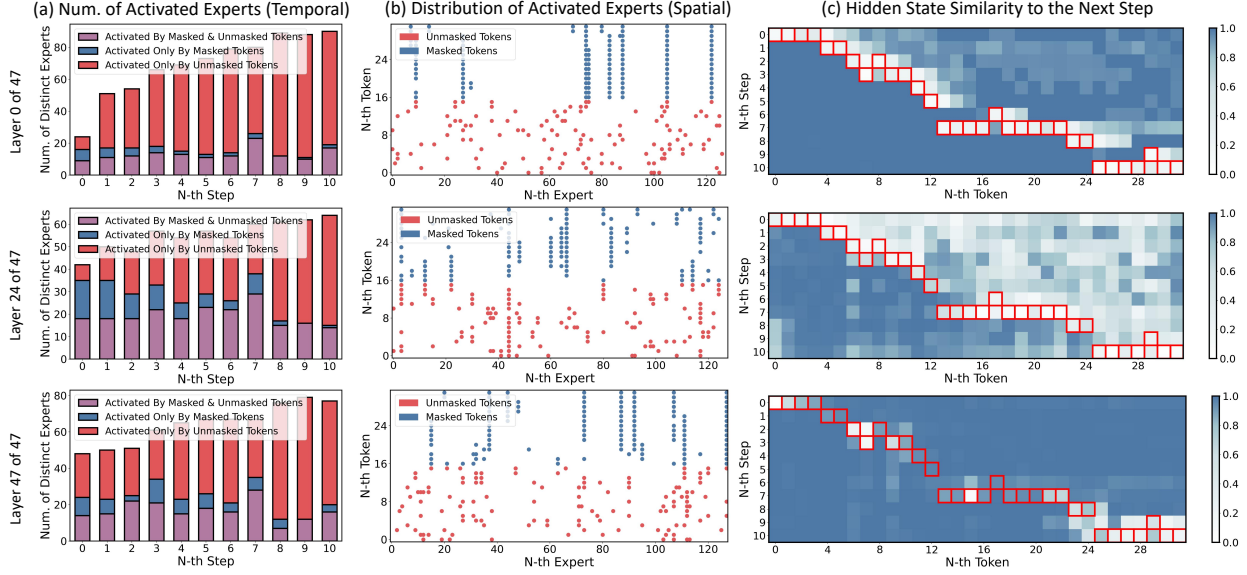


Figure 9. Temporal-spatial characteristics of expert activation and decoding with the SDAR 30B-A3B model on a prompt from the GSM8K dataset.

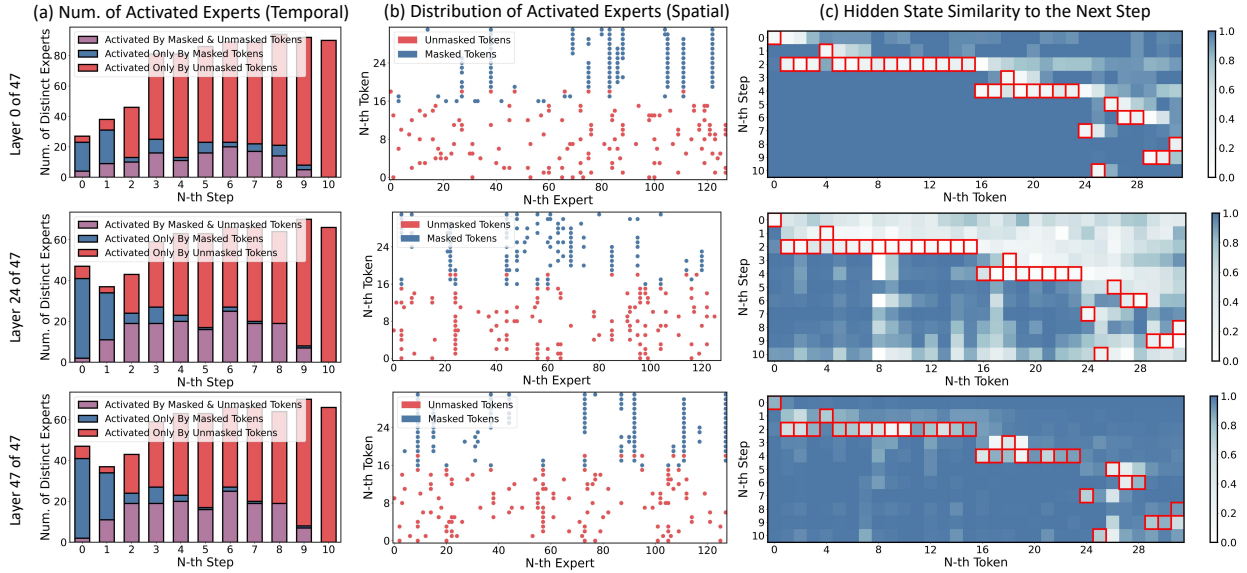


Figure 10. Temporal-spatial characteristics of expert activation and decoding with the SDAR 30B-A3B model on a prompt from the Math-500 dataset.

A.2. Expert activation with speculative exploration

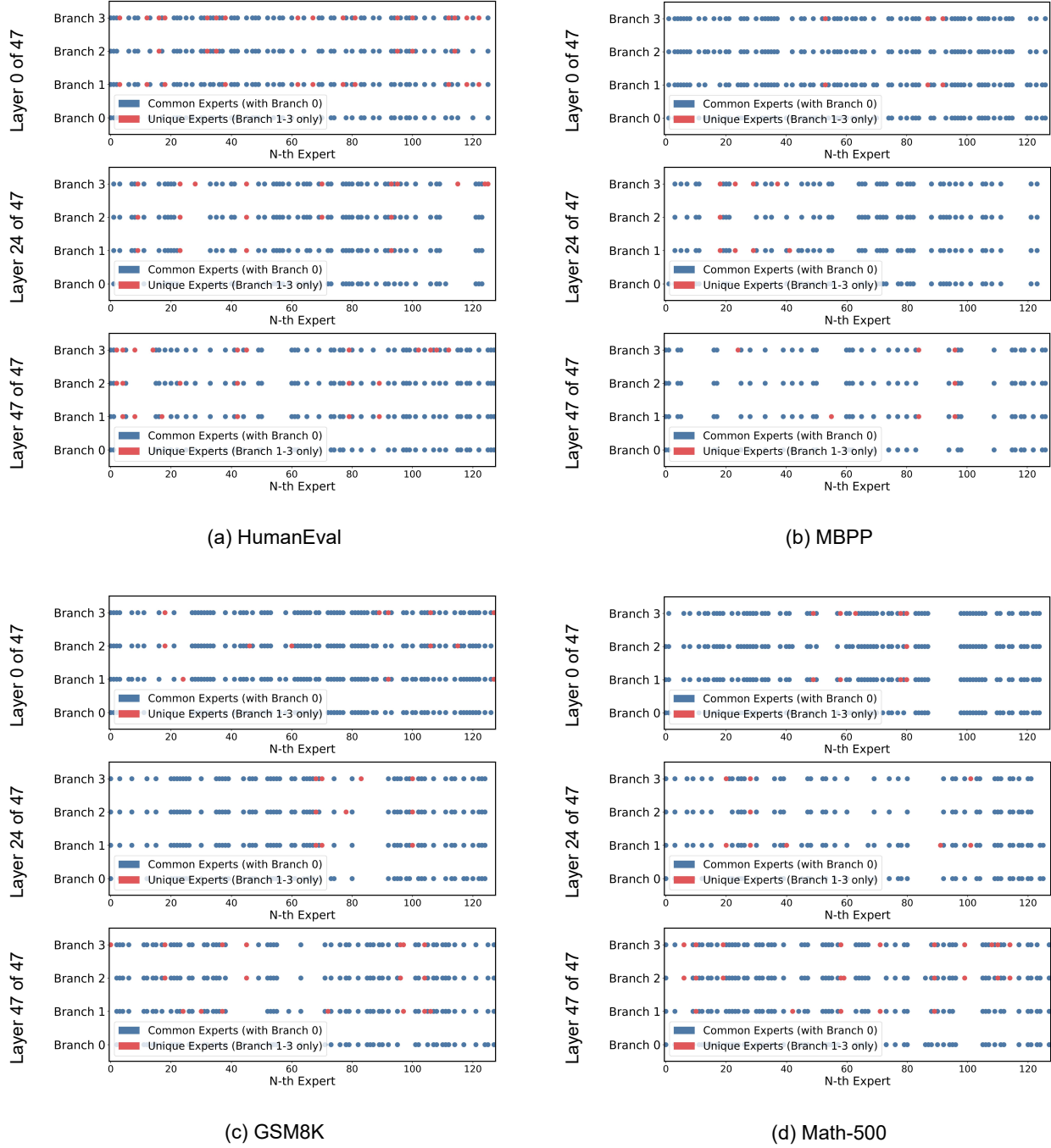


Figure 11. Expert activation with speculative exploration in SDAR 30B-A3B for responses from diverse benchmarks, measured at layer 0, 24 and 47 (of 47).

Experiments across diverse benchmarks show that, due to the high similarity among speculative branches, these branches tend to activate largely overlapping sets of experts, as shown in Figure 11. As a result, even when the effective decoding length increases by several times, aggressive speculative exploration introduces only a small number of additional experts, making multi-branch exploration particularly effective for accelerating MoE-based diffusion language models.

B. Implementation of Limited Activation for Cold Tokens

Here, we detail the computation pipeline of our Limited Activation strategy for cold tokens, which incorporates a specially designed double-round routing mechanism.

Algorithm 1 Limited Activation for Cold Tokens in SparseMoeBlock

Require: Hidden states $\mathbf{H}$, Past hidden states $\mathbf{H}_{past}$, Decoded index $\mathbf{D}$, Expert limit index $\mathbf{L}$, Number of experts N , Top-k value k

Ensure: Final hidden states $\mathbf{H}_{out}$, Router logits $\mathbf{R}$, Selected experts $\mathbf{E}_{sel}$

```

0:  $\mathbf{M}_{compute} \leftarrow \neg \mathbf{D}$  {Compute mask: tokens needing computation}
0:  $\mathbf{H}_{out}[\neg \mathbf{M}_{compute}] \leftarrow \mathbf{H}_{past}[\neg \mathbf{M}_{compute}]$  {Copy decoded tokens from cache}
0: if  $\exists i : \mathbf{M}_{compute}[i] = \text{True}$  then
0:    $\mathbf{H}_{comp} \leftarrow \mathbf{H}[\mathbf{M}_{compute}]$  {Extract tokens to compute}
0:   First-round Routing:
0:    $\mathbf{R}_{comp} \leftarrow \text{Gate}(\mathbf{H}_{comp})$  {Compute router logits}
0:    $\mathbf{W} \leftarrow \text{Softmax}(\mathbf{R}_{comp})$  {Routing weights}
0:    $\mathbf{W}_{topk}, \mathbf{E}_{sel} \leftarrow \text{TopK}(\mathbf{W}, k)$  {Initial top-k selection}
0:    $\mathbf{L}_{comp} \leftarrow \mathbf{L}[\mathbf{M}_{compute}]$  {Expert limit mask for compute tokens}
0:   if  $\exists i : \mathbf{L}_{comp}[i] = \text{True}$  then {Has cold tokens}
0:     Collect Necessary Experts from Hot Tokens:
0:      $\mathbf{I}_{hot} \leftarrow \{i : \mathbf{L}_{comp}[i] = \text{False}\}$  {Indices of hot tokens}
0:      $\mathbf{E}_{nec} \leftarrow \emptyset$  {Initialize necessary experts set}
0:     if  $|\mathbf{I}_{hot}| > 0$  then
0:        $\mathbf{E}_{nec} \leftarrow \bigcup_{i \in \mathbf{I}_{hot}} \mathbf{E}_{sel}[i]$  {Union of experts selected by hot tokens}
0:     end if
0:     Second-round Routing for Cold Tokens:
0:      $\mathbf{M}_{invalid} \leftarrow \{e : e \notin \mathbf{E}_{nec}, e \in [0, N)\}$  {Invalid expert mask}
0:      $\mathbf{W}[:, \mathbf{M}_{invalid}] \leftarrow 0$  {Zero out invalid experts}
0:      $\mathbf{W}_{topk}, \mathbf{E}_{sel} \leftarrow \text{TopK}(\mathbf{W}, k)$  {Re-select from necessary experts only}
0:   end if
0:   if norm_topk_prob then
0:      $\mathbf{W}_{topk} \leftarrow \mathbf{W}_{topk} / \sum_{j=1}^k \mathbf{W}_{topk}[:, j]$  {Normalize}
0:   end if
0:   Expert Computation:
0:    $\mathbf{H}_{comp}^{out} \leftarrow 0$ 
0:   for  $e = 0$  to  $N - 1$  do
0:      $\mathbf{I}_e \leftarrow \{(i, j) : \mathbf{E}_{sel}[i, j] = e\}$  {Tokens assigned to expert  $e$ }
0:     if  $|\mathbf{I}_e| > 0$  then
0:        $\mathbf{H}_{comp}^{out}[\mathbf{I}_e] += \text{Expert}_e(\mathbf{H}_{comp}[\mathbf{I}_e]) \odot \mathbf{W}_{topk}[\mathbf{I}_e]$ 
0:     end if
0:   end for
0:    $\mathbf{H}_{out}[\mathbf{M}_{compute}] \leftarrow \mathbf{H}_{comp}^{out}$ 
0:    $\mathbf{R}[\mathbf{M}_{compute}] \leftarrow \mathbf{R}_{comp}$ 
0: end if
0: return  $\mathbf{H}_{out}, \mathbf{R}, \mathbf{E}_{sel} = 0$ 

```
